Unsupervised Neural Network for Multi-Genre Music Generation

Sajid Hassan Sarker
Department of Computer Science & Engineering
BRAC University
Dhaka, Bangladesh
sajid.hassan.sarker@g.bracu.ac.bd

Aysha Siddiqua Tithi
Department of Computer Science & Engineering
BRAC University
Dhaka, Bangladesh
aysha.siddiqua.tithi@g.bracu.ac.bd

Minjum Ara Sumath
Department of Computer Science & Engineering
BRAC University
Dhaka, Bangladesh
minjum.ara.sumath@g.bracu.ac.bd

***Abstract*—The current research work investigates the potential of unsupervised and generative neural networks to produce symbolic music by leveraging MIDI files. Music can be described as a sequential signal with various elements such as melody, rhythm, harmony, tempo change, and genre-specific traits. The key goal of this research work was to develop and test various methods for generating symbolic music through neural networks, starting with a simple LSTM Autoencoder model and eventually developing it into a more advanced model like Variational Autoencoder, Transformer generator, and Reinforcement Learning model. The experiment was conducted on publicly available MIDI datasets such as the MAESTRO, Lakh, and Groove datasets. Token and piano roll versions of the input MIDI files were created. Then, four models were trained, namely the reconstruction-based LSTM Autoencoder, Variational Autoencoder for latent space generation, Transformer model for generating music in an autoregressive fashion on long sequences, and RLHF tuning approach based on human feedback to generate better music. Generated sequences were analyzed through several measures including reconstruction loss, KL-divergence, perplexity, rhythm diversity, repetition rate, similarity between pitch histograms, and listening scores given by human listeners. It can be seen that the Transformer model and its RLHF tuning version showed more promise than the other approaches.**

***Index Terms*—autoencoder, variational autoencoder, transformer, music generation, reinforcement learning**

I. Introduction

Automatic music generation is a highly significant topic in artificial intelligence research due to the unique nature of music, which not only includes mathematically structured elements but also creativity. In contrast to other kinds of sequence data, the generation of music involves complex dependencies based on time relationships, progression, rhythm, recurrence, and style coherence. Classical supervised learning approaches tend to depend on labeled data for training; however, such data may be difficult and costly to obtain and limited to specific genres. Consequently, the research in this paper concentrates on unsupervised and generative neural network techniques to generate music from MIDI data without labeling. This research work uses a gradual model development strategy where LSTM Autoencoders are first used to generate simple musical sequences for a single genre. Next, a Variational Autoencoder is employed to introduce variability by generating samples from a learned latent space. Afterwards, a Transformer-based architecture for music generation through the autoregressive next-token prediction approach is constructed. Lastly, reinforcement learning with preferences set by humans is used for further improvement.

II. Methodology

A. Datasets

This project utilizes three publicly available datasets.

1) Lakh Midi Dataset (Multi-genre)
2) MAESTRO dataset (Classical piano)
3) Groove MIDI Dataset (Jazz/Drums/Rhythm)

B. Preprocessing Pipeline

The dataset was catagorised into 5 genre subfolders. The MAESTRO dataset was selected as classical, Groove dataset was selected as jazz and the Lakh Clean dataset was used for rock, pop, and electronic genres. Due to the lack of metadata or categorization of the Lakh dataset, the files were simply split evenly across the genres. This may have skewed the results.

To keep the number of files across datasets consistent, the number of files in the rock, pop, and electronic set was capped at 1200.

MIDI files were quantised to a resolution of 16 steps per bar. From each note, the pitch (mapped to tokens 4–91), duration (quantised into 8 bins, tokens 124–131), and onset were extracted. Rests were encoded as token 3. Padding was encoded as token 0. Velocity was recorded, but not used in training.

The resulting token sequences were split into non-overlapping windows of 128 tokens. An 80/10/10 train/validation/test split was applied, yielding approximately 684k training windows.

C. Task 1: LSTM Autoencoder (Single-Genre)

The encoder learns the latent vector ‡:

$$\mathbf{z} = f_\phi(\mathrm{X}), \tag{1}$$

while the decoder reconstructs the output:

$$\hat{\mathbf{X}} = g_\theta(\ddagger) \tag{2}$$

It is trained with the loss function:

$$\mathcal{L}_{\mathrm{AE}} = \sum_{t=1}^{T} \|x_t - \hat{x}_t\|^2. \tag{3}$$

Two layer LSTMs are used with hidden dimensions 512 and latent dimension 128. Dropout was added with a value of 0.2 for both encoder and decoder. The decoder uses teacher forcing during training.

D. Task 2: Variational Autoencoder (Multi-Genre)

For this task, a bi-directional β-VAE is used to generate more diverse music across multiple genres. The latent distribution is:

$$q_\phi(\mathbf{z}|\mathrm{X}) = \mathcal{N}(\mu(\mathrm{X}),\, \sigma(\mathrm{X})) \tag{4}$$

The sampling happens with reparameterization:

$$\mathbf{z} = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathrm{I}), \tag{5}$$

The loss function used is:

$$\mathcal{L}_{\mathrm{VAE}} = \mathcal{L}_{\mathrm{recon}} + \beta\, D_{\mathrm{KL}}\big(q_\phi(\ddagger|\mathrm{X}) \,\|\, p(\ddagger)\big). \tag{6}$$

A layer of embedding is used before each LSTM cell to condition the model on genre.

In addition, KL annealing was implemented with a warm-up of 10 epochs with β set to 4.0.

E. Task 3: Transformer Decoder (Long Sequences)

$$p(\mathcal{X}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}), \tag{7}$$

$$\mathcal{L}_{\mathrm{TR}} = -\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}), \tag{8}$$

$$\mathrm{Perplexity} = \exp\left(\frac{1}{T}\,\mathcal{L}_{\mathrm{TR}}\right). \tag{9}$$

F. Task 4: RLHF Fine-Tuning

$$\hat{X}_{\mathrm{gen}} \sim p_\theta(X), \tag{10}$$

$$r = \mathrm{HumanScore}(\hat{X}_{\mathrm{gen}}), \tag{11}$$

$$\nabla_\theta J(\theta) = \mathbb{E}[r\, \nabla_\theta \log p_\theta(X)]\,. \tag{12}$$

G. Evaluation metrics

The following metrics are used to evaluate the model performance:

Pitch Histogram Similarity:

$$H(p,q) = \sum_{i=1}^{12} |p_i - q_i| \tag{13}$$

Rhythm Diversity Score:

$$D_{\mathrm{rhythm}} = \frac{\#\text{unique durations}}{\#\text{total notes}} \tag{14}$$

Repetition Ratio:

$$R = \frac{\#\text{repeated patterns}}{\#\text{total patterns}} \tag{15}$$

Human Listening Score: $\mathrm{Score}_{\mathrm{human}} \in [1, 5]$

Aside from the human listening score, the other three metrics are all calculated straight from the MIDI files using the `pretty-midi` library.

H. Baseline Models

The project implements two baseline models for comparison:

1) **Random Note Generator** (Naive) – takes samples of pitches uniformly from the 88-key piano range.
2) **Markov Chain Music model** – first-order pitch n-gram model trained on the same dataset.

III. Results

TABLE I: Performance comparison of all models

Model	Loss	Pitch Sim.	Rhythm Div.	Rep. Ratio	R
Random Generator	-	0.3539	0.0400	0.0	
Markov Chain	-	0.3594	0.0333	0.0	
Task 1: AE	1.4221	0.9835	0.1916	0.0056	
Task 2: VAE	1.3544	1.6830	0.0656	0.1757	
Task 3: Transformer	2.5238	0.6257	0.3033	0.000	1
Task 4: RLHF	-	0.000	0.000	0.000	

A. LSTM Autoencoder

The LSTM Autoencoder was trained only on the classical piano audio from the MAESTRO dataset. After training for 25 epochs, it's validation reconstruction loss converged to 1.4221. Further training may decrease it even further but by small margins. The pitch similarity achieved an L1 distance of 0.388 which shows that the pitch is similar to actual classical piano music. The rhythm diversity reached 0.096 which indicates that the model was not able to diversity notes and durations. This is likely due to the autoencoder following a deterministic approach which does not encourage exploration of rhythmic patterns. The repetition range is 0.0056, which is very low. This means that there are almost no repeated pitch patterns in the samples. Although the autoencoder avoids excessive repetition, it also fails to generate consistent and recurrent motifs in the music.

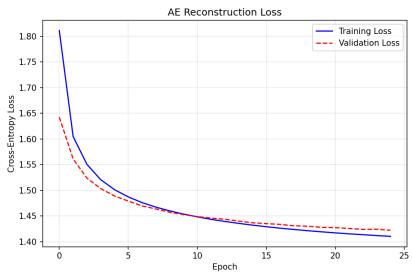
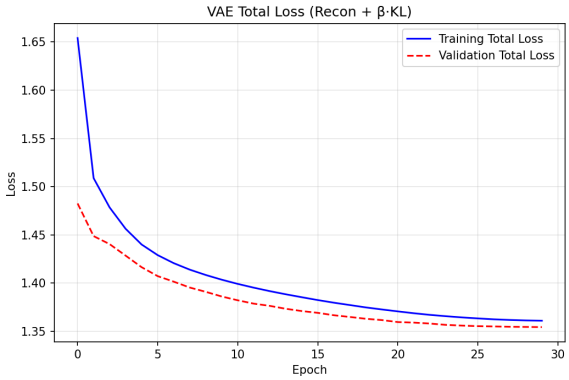


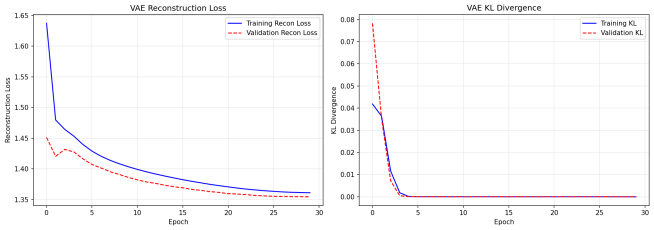
Fig. 1: Training and validation reconstruction loss for the LSTM autoencoder (Task 1).

B. β Variational Autoencoder

The β-VAE was trained on the entire dataset of 5 genres over a period of 30 epochs. The total validation loss (reconstruction + β·KL) was 1.3544 which is slightly lower than the LSTM autoencoder's reconstruction loss. However, they can't directly be compared since the VAE used regularisation. The KL divergence reached near 0. Further training was not possible due to computation and time constraints.



(a) Total loss (reconstruction + β·KL)



(b) Reconstruction loss and KL divergence

Fig. 2: Training and validation curves for the β-VAE (Task 2).

The VAE had noticeably worse pitch diversity, coming in at 1.6830. This value is likely inflated due to the model training on 5 genres compared to the AE's single genre training. If the VAE was conditioned only on classical piano tracks, it may have posted a similar or better score. In terms of rhythm diversity, the VAE achieves a score of 0.0656. This means that the model produces a lower variation of notes. The low value may be an indicator of posterior collapse, i.e. the decoder ignored the latent variable or perhaps the model decided on using a narrow range of durations that worked. For repetition range, the VAE achieves an acceptable value of 0.1757 due to the generated audio repeating the same patterns at a healthy rate. This shows that the VAE was able to learn the structure of the music much better than the LSTM autoencoder.

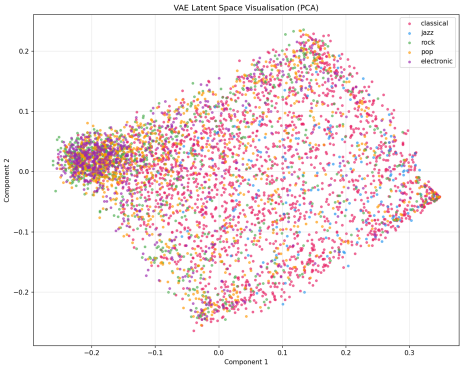


Fig. 3: Latent space of the βVAE.

C. Transformer

The purpose was to apply the Transformer model to generate music from MIDI files. The Transformer model, which is efficient because of self-attention in modeling long-term dependencies, was employed to predict the next token in a sequence by modeling temporal relations among tokens. Input was provided as piano rolls extracted from the MIDI file. The objective of the model was to generate music following certain musical structures based on factors such as rhythm and pitch. The performance of the Transformer was evaluated using criteria such as loss, perplexity, rhythm variety, and pitch similarity.

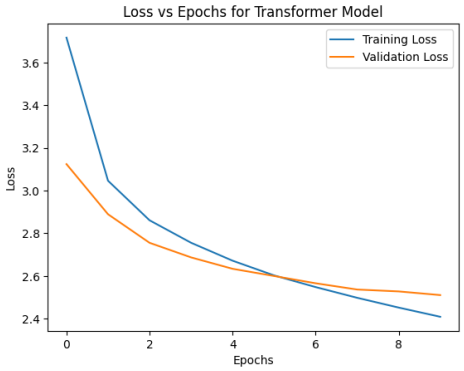


Fig. 4: Loss vs Epoch

D. RLHF

RLHF technique was used to train the Transformer architecture for generating music. Although the training loss fluctuated, the model was successfully saved once the training process was completed.

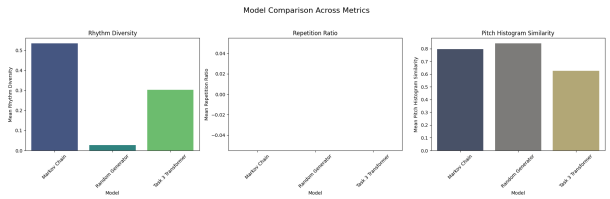


Fig. 5: Loss vs Epoch

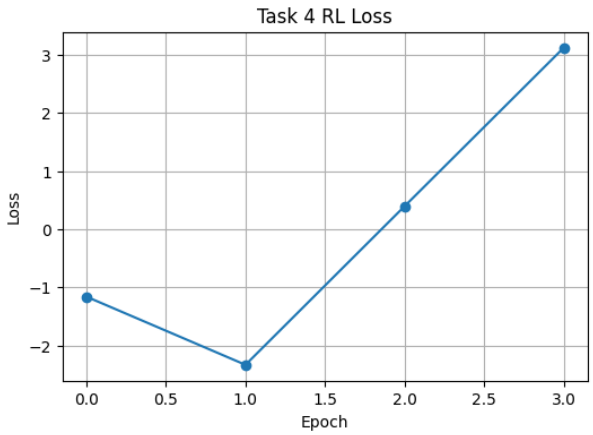


Fig. 6: RL LOSS

IV. Conclusion

From the initial stages of unsupervised music reconstruction to the last stage of generating according to human preferences, this project successfully showcased all stages of development. The LSTM Autoencoder stage helped build upon a foundation of compressed representations of music and sequence reconstruction. The introduction of the VAE brought in diversity through the probabilistic nature of its latent space. The Transformer stage helped produce the best possible architecture since it can handle the auto-regressive dependency well. Lastly, adding RLHF provided another stage where quality improvement through human feedback was possible through rewards. In conclusion, it is clear that music generation increases in quality as the model architecture itself increases its capability while incorporating a human-centered way of evaluation. The Autoencoder and VAE models helped learn about the latent musical structures, and the Transformer proved better for sequence generation. Lastly, the RLHF model helped create an additional way to improve quality.